

CASM: CONTEXT-AWARE SEMI-MARKOV POST-PROCESSOR FOR BEAT TRACKING

Zhanhong He^{1,3}, Hanyu Meng^{1,3}, Yaolong Ju^{1,*}

¹Great Bay University, Dongguan, China

²The University of Western Australia, Perth, Australia

³The University of New South Wales, Sydney, Australia

ABSTRACT

At the final stage of music beat tracking, neural activations need to be decoded into a discrete, musically coherent event sequence. Direct peak picking follows the local evidence but tends to produce spurious and missed detections. Dynamic Bayesian networks (DBNs), the dominant post-processing approach, suppress such errors by imposing explicit human-specified global assumptions. They work well when predefined tempo and meter assumptions match the music, but expressive or rhythmically complex pieces often depart from them, resulting in overrides correct local evidence instead of correcting it. We address this limitation by proposing CASM, a model-agnostic, context-aware semi-Markov decoder that exploits local structure in beat activations. CASM estimates local periodicity and ambiguity from the activations, then commits strongly when the cues are clear and relaxes toward the neural output when they are ambiguous, with built-in safeguards keeping this behavior stable and predictable. Using a single configuration, CASM improves F1, CMLt, and AMLt across BeatThis, MSCNN, and TCN models on GTZAN, and delivers state-of-the-art performance on SMC. Further analysis shows that CASM is less sensitive than DBNs to the choice of calibration data.¹

Index Terms— beat tracking, context-aware, semi-Markov model, post-processing, local periodicity

1. INTRODUCTION

Beats give music its temporal scaffold, so reliable beat tracking is crucial for music production, analysis, synchronization, and interactive systems [1]. More recently, beat tracking has become a key component of multimodal AI-generated content (AIGC), where the estimated beats serve as timing anchors for aligning generated content such as dance [2, 3], video [4, 5], and music [6, 7]. Current music beat trackers typically operate in two stages: a network maps audio to framewise beat activations, and a post-processor converts these activations into discrete events. Despite recent attempts to remove the post-processor [8–10], many contemporary systems [11–15] still rely on it to obtain musically coherent estimates. However accurate the neural activations become, direct peak picking makes each decision locally without considering the periodicity of beat sequences. Structured post-processing addresses this limitation by combining local activation evidence with music temporal regularity.

For post-processors, the central design question is when and how strongly this temporal regularity should constrain the local evidence. Weak temporal regularization preserves the network’s predictions but can sacrifice continuity, whereas overly strong regularization can override correct evidence under expressive or unusual timing. Earlier methods largely set this balance through predefined temporal constraints. The pioneering dynamic-programming approach [16] penalized inter-beat intervals (IBIs) that deviate from a single global tempo. Conditional random fields (CRFs) [17] provided a more flexible probabilistic formulation that allowed local tempo variations while still penalizing abrupt IBI changes. The dominant post-processing approach is the dynamic Bayesian network (DBN) [18], which operates under pre-specified tempo ranges, meters, and transition behaviors. This works well when those assumptions match the recording, but may require corpus-specific retuning when they do not [19].

PLPDP [20] represents an important step toward locally adaptive post-processing. It converts predominant local pulse (PLP) into a time-varying IBI $\hat{\delta}(n)$ and confidence $\lambda(n)$: the former centers the tempo-consistency penalty, while the latter scales it according to the mean heights of adjacent PLP peaks. PLPDP therefore already adapts both the target and strength of the temporal constraint. Its confidence, however, is a non-comparative measure: the dynamic-programming condition contains no explicit term for how decisively the selected periodicity exceeds a competing interpretation. The relative uncertainty between, for example, half- and double-tempo hypotheses can thus remain implicit when they receive similar local support.

We introduce CASM, a context-aware semi-Markov post-processor that makes this comparison explicit. At each retained activation maximum, CASM scores the admissible autocorrelation lags, selects the strongest as the local period, and computes a normalized margin to the strongest alternative outside a small lag neighborhood. This margin is not a calibrated probability and does not create a multi-hypothesis tempo path; instead, through a multiplicative weight and an ambiguity-dependent scale, it controls the stiffness of the duration cost centered on the selected period. CASM combines the period and margin at both endpoints of each candidate-to-candidate segment. A clear winner yields a strong duration preference that can recover weak but activation-supported beats, whereas a small margin flattens that preference so activation evidence dominates while periodicity remains unresolved. Fig. 1 illustrates these two regimes. A deterministic beat-count safeguard and beat-synchronous meter stage

*Corresponding author, juyaolong@gbu.edu.cn.

¹Code and demo are available at <https://zhanh-he.github.io/casm-beat-tracking>.

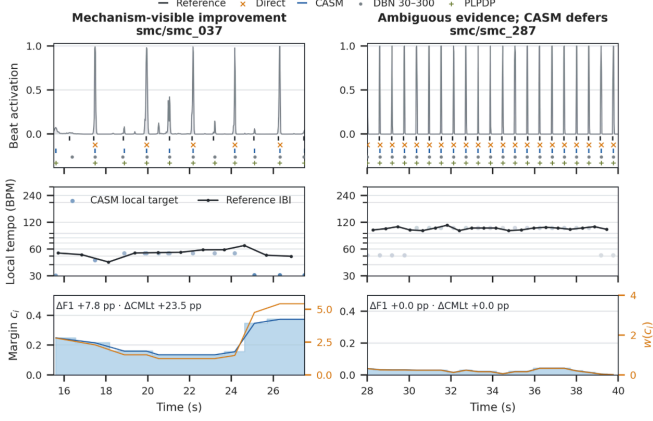


Fig. 1. CASM behaviour on held-out Beat This activations from SMC. Clear local periodicity recovers weak activation-supported beats (left), whereas ambiguity among competing periods weakens the duration preference, allowing the decoded path to match Direct in this example (right). Reference IBI is shown only for interpretation.

further stabilizes the output. CASM is model-agnostic, requires no neural-network retraining, and is provided as a pip-installable reference implementation with a madmom-style activation interface.

2. METHODOLOGY

2.1. CASM Overview

The proposed Context-Aware Semi-Markov (CASM) decoder is a model-agnostic post-processor that operates on beat and downbeat activations produced by a frozen tracker. It modifies only the decoding stage and therefore requires no retraining. Like PLPDP, CASM estimates a local beat period and its support from the activation sequence. CASM differs in how this support is defined and used: it measures the separation between the dominant period and competing tempo hypotheses and uses this margin to adapt the rhythmic constraint. Consequently, CASM enforces local regularity when the periodic evidence is clear, but relaxes the constraint when the evidence is ambiguous.

2.2. Context-aware beat decoding

A frozen tracker produces a beat logit z_t^b at each frame t , with activation $a_t = \text{sigmoid}(z_t^b)$ and frame rate F . We retain local maxima above a fixed threshold θ_c as candidates $\mathcal{C} = \{t_1, \dots, t_N\}$, assigning each candidate a clipped evidence score e_i . For $i < j$, the elapsed time is $\Delta_{ij} = (t_j - t_i)/F$, and edge (i, j) is admissible when $60/\Delta_{ij}$ lies within 30–300 BPM. CASM selects an ordered candidate path $\pi = (i_1, \dots, i_K)$, with $i_1 < \dots < i_K$, by maximizing

$$\mathcal{S}(\pi) = \sum_{k=1}^K e_{i_k} - \sum_{k=2}^K D_{i_{k-1}, i_k} \quad (1)$$

where D_{ij} is the duration cost between candidates i and j . The two terms respectively reward strong beat evidence and penalize rhythmically inconsistent intervals. Because each

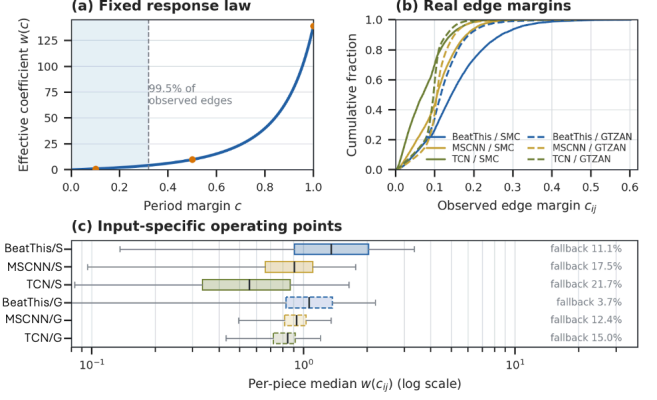


Fig. 2. Input-conditioned duration stiffness under one frozen CASM configuration. (a) The fixed constants map period margin c to coefficient $w(c)$; shading contains 99.5% of observed structured-path edges. (b) Edge margin distributions differ across backbones and corpora. (c) Per-piece median coefficients therefore occupy different operating ranges; annotations give the separate count-ratio fallback rate. Fixed parameters define one response law, not one effective rigidity for every input.

transition scores a variable-duration interval, CASM has a semi-Markov structure.

At candidate i , $q_i(\ell)$ denotes the normalized autocorrelation score at an admissible lag ℓ within a window centred on t_i , including a mild fast-tempo prior. The dominant lag $\ell_i^* = \arg \max_{\ell} q_i(\ell)$ gives the local period $\tau_i = \ell_i^*/F$. We define the strongest competing hypothesis outside a ± 2 -frame neighbourhood as $q_i^{\text{alt}} = \max_{\ell: |\ell - \ell_i^*| > 2} q_i(\ell)$. The normalized period-separation margin is

$$c_i = \text{clip}_{[0,1]} \left(\frac{q_i(\ell_i^*) - q_i^{\text{alt}}}{|q_i(\ell_i^*)| + \epsilon_c} \right) \quad (2)$$

where $\epsilon_c > 0$ stabilizes the denominator. A large c_i indicates a clearly dominant period, whereas a small value indicates ambiguity, including half- or double-time alternatives. Thus, c_i is a deterministic separation margin rather than a calibrated probability. Both τ_i and c_i are median-filtered over five consecutive candidates.

For edge (i, j) , CASM combines the endpoint estimates as $\bar{\tau}_{ij} = \sqrt{\tau_i \tau_j}$ and $c_{ij} = \sqrt{c_i c_j}$. The ambiguity-conditioned constraint strength is $g(c) = \lambda c / \{2[\sigma_0 + (1 - c)\sigma_u]^2\}$, giving the duration cost

$$D_{ij} = g(c_{ij}) \log^2 \left(\frac{\Delta_{ij}}{\bar{\tau}_{ij}} \right) \quad (3)$$

Here, $\lambda > 0$ controls the overall penalty strength, $\sigma_0 > 0$ sets the minimum timing tolerance, and $\sigma_u \geq 0$ controls relaxation under ambiguity. The cost vanishes when $\Delta_{ij} = \bar{\tau}_{ij}$; a large c_{ij} enforces rhythmic regularity, while a small c_{ij} relaxes the constraint. CASM can therefore recover weak but rhythmically supported beats without forcing an unreliable tempo grid.

Figure 2 shows how Eq. (3) converts the local period-separation margin into an input-dependent timing constraint. Panel (a) shows the fixed response, panel (b) maps transi-

tions from different datasets and backbones onto it, and panel (c) summarizes the resulting track-level constraint strengths. Thus, one frozen CASM configuration adapts its constraint across inputs. The figure illustrates decoder conditioning, not tracking accuracy; the direct fallback is a separate safeguard.

2.3. Safeguards and downbeat decoding

A Viterbi-style dynamic program maximizes Eq. (1) and backtracks the best beat path, allowing a restart after an unfavourable prefix. As a safeguard, CASM also forms a direct fallback path using seven-frame max pooling, a zero-logit threshold, and one-frame deduplication [9]. If $|\pi_{\text{CASM}}|/|\pi_{\text{direct}}| \notin [r_{\min}, r_{\max}]$, indicating an implausibly sparse or dense CASM path, the direct path is returned. This rule is deterministic and requires no learned router.

Downbeats are decoded on the selected beat grid using a second dynamic program that favours valid bar lengths and discourages unnecessary meter changes. If its output strongly disagrees with the direct downbeat activations snapped to the same grid, the snapped result is used, ensuring that every downbeat coincides with an output beat. All scalar parameters are calibrated once on the development folds and then frozen across tracks, datasets, and backbone models. Inference requires no reference annotations, externally supplied BPM or meter, per-track tuning, or learned decoder parameters.

3. EXPERIMENTS

3.1. Evaluation protocol

Datasets and splits. We use the publicly released data from BeatThis [9], which contains 4,556 tracks from 18 beat-tracking datasets, including SMC, Hainsworth, Ballroom, Beatles, and others. This data collection provides preprocessed log-Mel spectrograms computed from 22.05kHz mono audio at 50 fps, together with a fixed eight-fold partition to conduct cross-validation experiments. For each fold, the corresponding backbone checkpoint is trained on the remaining seven folds, ensuring that evaluated tracks remain unseen during training. Among these datasets, SMC provides beat but not downbeat annotations, and is widely recognized as the hard one compared to the others. Additionally, GTZAN provides 993 tracks as the held-out evaluation set, which is excluded from the cross-validation pool.

Backbones. We select three models whose implementations are publicly available and readily reproducible. For BeatThis, we use the original code and pretrained checkpoints [9]. MSCNN is adapted from the multi-scale, multi-task convolutional network of [21], which was designed for 60s Bark-scale specific-loudness features. We modify it to accept the 30s the unified log-Mel input and remove the task heads unrelated to beat and downbeat tracking. TCN was introduced by [22], and we use the re-implementation distributed with [21], which has been adapted from its original 5s input to the same 30s log-Mel input, with the tempo task head removed. All resulting backbones are trained on the same data and frozen before decoding performance comparison, ensuring that every post-processor receives exactly the same 50 Hz frame-wise beat/downbeat activations from a given backbone.

Evaluation. Following previous work [9], the eight fold-specific checkpoints of each backbone are evaluated on

GTZAN and their corresponding out-of-fold SMC partitions, with scores macro-averaged over total evaluation. We use `mir_eval` to compute F1 score with a ± 70 ms tolerance, CMLt, and AMLt [23]. CMLt measures continuity at the correct metrical level, whereas AMLt additionally accepts standard off-beat and half- or double-level interpretations.

3.2. Post-processors and tempo support

As we defined our task is offline, the future information is accessible for model to top up the estimate accuracy. We compare CASM with the direct decoding, along with other offline post-processing methods including DBN [18], PLPDP [20], and CRF beat detection [24]. Direct is the peak-picking path defined in Sec. 2, and CASM uses its fixed 30–300-BPM support. PLPDP, CombFilter, and CRF are run with their released/default parameters. For methods without a joint beat/downbeat decoder, downbeats are decoded from the downbeat activation stream and snapped to the corresponding beat grid.

DBN receives separate treatment because its behaviour is particularly sensitive to global tempo support. We use the standard joint madmom configuration [18]: meters $\{3, 4\}$, transition weight 100, observation weight 16, threshold 0.05, and the default 55–215-BPM range. Since many SMC data are fall below 55 BPM [19], Table 1 also explores a 30–300-BPM DBN variant in which only the tempo range is changed. For a matched diagnostic, we additionally report CASM with its support restricted to 55–215 BPM on BeatThis activations, with all other CASM settings remain unchanged.

3.3. CASM calibration

CASM has no learned weights or random seed. The final configuration used in all main tables is calibrated once on the union of SMC folds 1–7 (7F) and then frozen for every track, backbone, and dataset. The same global scalar settings are therefore used on BeatThis, MSCNN, and TCN, on both SMC and GTZAN, without per-track or per-dataset BPM, meter, or parameter tuning at inference. Because the aggregate SMC results use eight-fold out-of-fold backbone activations while the decoder configuration is calibrated on folds 1–7, they are out-of-fold with respect to backbone training but are not a nested estimate of decoder calibration.

We analyse calibration-data sensitivity separately rather than using it to choose the main operating point. SMC fold 0 is held out from decoder calibration throughout. From folds 1–7, we enumerate every 1-, 2-, and 4-fold calibration subset (7, 21, and 35 combinations) and the seven-fold union. For both CASM and DBN, a decoder configuration is selected independently on each calibration subset and then evaluated on the same fixed SMC-fold-0 panel and on GTZAN using one fixed backbone seed. The resulting distributions therefore measure sensitivity to the amount and composition of calibration data, while the 7F CASM configuration is the maximally supported global configuration used for the main results.

Table 1. Post-processing comparison on fixed backbone activations. Backbone-only rows use direct peak picking. Post-processors use 30–300 BPM unless marked ^[dbn], which denotes the 55–215 BPM range optimized for DBN in GTZAN-like music.

Method	GTZAN (Beat / Downbeat)						SMC (Beat)		
	F1	CMLt	AMLt	F1	CMLt	AMLt	F1	CMLt	AMLt
BeatThis [9]	89.1	79.8	89.8	78.3	67.2	79.1	62.7	51.4	61.0
w. CASM	+0.3	+0.8	+1.2	+0.7	+3.6	+8.0	+0.3	+2.4	+3.6
w. CASM ^[dbn]	+0.1	+0.5	+0.6	+0.5	+2.9	+5.8	+0.1	+0.4	+2.0
w. DBN [18]	−0.5	−0.9	+0.2	−1.5	−0.6	+1.1	−1.6	+0.4	+3.5
w. DBN ^[dbn]	−1.0	+0.7	+1.3	−0.9	+6.2	+8.7	−5.2	−3.9	+1.5
w. PLPDP [20]	+0.1	+0.3	+0.3	+0.1	+0.2	+0.2	−0.9	−3.5	−0.3
w. CRF [24]	−0.6	−0.3	+1.4	−1.4	+1.9	+5.5	−1.2	+1.4	+1.2
MSCNN [21]	86.1	73.4	79.9	70.8	53.1	63.1	57.2	42.0	48.4
w. CASM	+0.4	+1.6	+2.4	+1.5	+12.5	+18.3	+0.3	+3.5	+6.8
w. DBN	−1.1	+0.2	+5.9	−0.9	+9.5	+14.6	−0.2	+2.2	+4.1
w. DBN ^[dbn]	−0.6	+4.0	+7.2	+1.8	+15.8	+21.2	−4.9	+0.3	+3.9
TCN [22]	87.1	73.8	81.7	62.4	24.6	59.9	52.9	30.1	36.0
w. CASM	+0.3	+1.3	+1.6	+2.1	+9.2	+13.4	+1.7	+4.5	+8.2
w. DBN	−1.9	−0.8	+4.4	−0.2	+8.9	+9.3	−0.4	+2.1	+5.9
w. DBN ^[dbn]	−2.6	−0.7	+5.0	+4.9	+23.2	+22.9	−0.9	−7.0	+2.3

Table 2. Comparison with recent beat-tracking systems. BeatThis (direct) and CASM results were obtained from our experiments, while all other values are reported from their original papers.

Method	GTZAN (Beat / Downbeat)						SMC (Beat)		
	F1	CMLt	AMLt	F1	CMLt	AMLt	F1	CMLt	AMLt
HingeNet-MERT w. DBN [12]	89.7	81.4	94.3	77.4	71.6	87.2	61.7	—	—
HingeNet-MusicFM w. DBN	89.2	80.9	93.7	79.8	73.2	89.5	60.6	—	—
BeatFM-MERT w. DBN [13]	89.5	82.7	93.9	76.7	70.2	85.4	61.3	—	—
BeatFM-MusicFM w. DBN	89.1	80.6	93.5	79.6	74.4	88.7	60.5	—	—
BeatMamba w. DBN [14]	88.9	82.3	94.3	—	—	—	58.7	46.8	62.4
BeatKAN w. DBN [15]	88.2	78.1	92.3	—	—	—	59.8	48.4	64.0
BeatThis-MDM [25]	89.7	82.9	92.5	79.5	76.4	88.5	61.0	48.9	64.3
BeatThis [9]	89.1	79.8	89.8	78.3	67.2	79.1	62.7	51.4	61.0
BeatThis w. CASM	89.4	80.6	91.0	79.0	70.8	87.1	63.0	53.8	64.6

4. RESULTS

4.1. Comparison with post-processors

Table 1 isolates the post-processor by holding the backbone activations fixed. With 30–300-BPM support, the CASM improves BeatThis (direct pick-peaking) on every reported F1, CMLt, and AMLt entry across all three backbones and both datasets. Relative to DBN, CASM generally preserves F1 better, while DBN can produce larger continuity gains, particularly for GTZAN downbeats. The large SMC change between the 55–215- and 30–300-BPM DBN variants also shows how strongly a fixed DBN can depend on its global tempo range. PLPDP, CombFilter, and CRF provide useful alternatives, but none improves every reported metric under the same fixed activations. Overall, CASM gives a conservative F1–continuity operating point that transfers across backbone families without changing decoder parameters.

4.2. Comparison with the state of the art

Table 2 compares the BeatThis works with CASM results with recent literature. While using CASM improves six beat/downbeat metrics compared to BeatThis (directly peak-

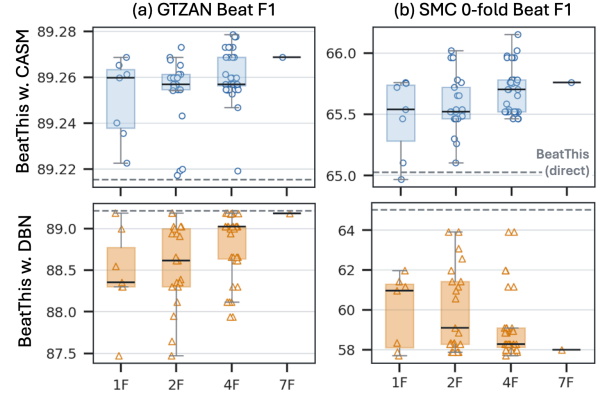


Fig. 3. Sensitivity to calibration data for CASM and DBN. Each point corresponds to a decoder configuration selected from one of the 7/21/35 exhaustive 1-/2-/4-fold subsets of SMC folds 1–7 and evaluated on fixed SMC-fold-0 and GTZAN panels; 7F is a single configuration. Boxes are descriptive because calibration subsets overlap, and dashed lines show Direct performance.

picking) on the GTZAN, we reported the highest scores on SMC compared to the other SOTA systems. On GTZAN, stronger task-specific or pre-trained frontends retain the best individual scores. The literature table therefore provides system-level context; controlled claims about decoder behaviour come from Table 1, where the backbone activations are held fixed.

4.3. Sensitivity to calibration data

The sensitivity experiment keeps the evaluation panels fixed and changes only the data used to calibrate the decoder. For each calibration scale $k \in \{1, 2, 4, 7\}$, CASM and DBN are calibrated on the corresponding SMC fold subsets described in Sec. 3, and every selected configuration is evaluated on the same held-out SMC fold 0 and the same GTZAN backbone seed. Thus, the spread at a given scale measures dependence on which calibration folds were used, while the movement of the distribution centre measures how additional calibration data changes transfer to the two fixed evaluation panels.

Fig. 3 shows a clear difference in calibration behaviour. CASM remains tightly clustered across fold combinations, and its central performance improves consistently as more calibration folds are used on both the held-out SMC panel and GTZAN. In contrast, DBN exhibits much wider variation across calibration subsets, and the trends on SMC and GTZAN do not move together: configurations favoured by one panel can transfer poorly to the other. This cross-dataset disagreement is consistent with stronger calibration-set over-specialisation in the fixed DBN parameterisation. The 7F CASM setting used in the main tables is therefore supported by the full available calibration pool rather than selected from a favourable smaller-fold combination.

5. CONCLUSION AND FUTURE WORK

CASM revisits structured beat decoding as an inductive bias conditioned on each input, not as a return to a fixed global grid. It joins activation-supported events with locally centred segment potentials and deliberately relaxes those potentials when periodic evidence is ambiguous. Once globally calibrated, the same deterministic module runs without upstream retraining, user annotation, or per-track tempo and meter settings. Current evidence supports a conservative F1-continuity alternative to Direct and DBN decoding, not universal dominance. The next version should use strict nested or leave-one-dataset-out calibration, matched tempo-support baselines, PLPDP and mechanism ablations, and explicit safeguard statistics. Multi-hypothesis period paths, causal decoding, and a differentiable duration layer are natural extensions, as is evaluation on every recent tracker that releases compatible activations.

Acknowledgments. The computational resources are supported by SongShan Lake HPC Center (SSL-HPC) at Great Bay University.

6. REFERENCES

- [1] Hendrik Purwins, Bo Li, Tuomas Virtanen, Jan Schlüter, Shuo-Yiin Chang, and Tara Sainath, “Deep learning for audio signal processing,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 13, no. 2, pp. 206–219, 2019.
- [2] Jonathan Tseng, Rodrigo Castellon, and C. Karen Liu, “EDGE: Editable dance generation from music,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 448–458.
- [3] Zikai Huang, Xuemiao Xu, Cheng Xu, Huaidong Zhang, Chenxi Zheng, Jing Qin, and Shengfeng He, “Beat-It: Beat-synchronized multi-condition 3D dance generation,” in *Proceedings of the European Conference on Computer Vision (ECCV)*. 2024, pp. 273–290, Springer.
- [4] Jiashuo Yu, Yaohui Wang, Xinyuan Chen, Xiao Sun, and Yu Qiao, “Long-term rhythmic video soundtracker,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023, vol. 202 of *Proceedings of Machine Learning Research (PMLR)*, pp. 40339–40353.
- [5] Xinyi Tong, Yiran Zhu, Jishang Chen, Chunru Zhan, Tianle Wang, Sirui Zhang, Nian Liu, Tiezheng Ge, Duo Xu, Xin Jin, Feng Yu, and Song-Chun Zhu, “Video echoed in music: Semantic, temporal, and rhythmic alignment for video-to-music generation,” in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026, vol. 40, pp. 25983–25991.
- [6] Yun-Han Lan, Wen-Yi Hsiao, Hao-Chung Cheng, and Yi-Hsuan Yang, “MusiConGen: Rhythm and chord control for transformer-based text-to-music generation,” in *Proc. ISMIR*, 2024.
- [7] Ke Chen, Yusong Wu, Haohe Liu, Marianna Nezhurina, Taylor Berg-Kirkpatrick, and Shlomo Dubnov, “MusicLDM: Enhancing novelty in text-to-music generation using beat-synchronous mixup strategies,” in *Proc. ICASSP*, 2024, pp. 1206–1210.
- [8] Tsung-Ping Chen and Li Su, “Toward postprocessing-free neural networks for joint beat and downbeat estimation,” in *Proc. ISMIR*, 2022, pp. 27–35.
- [9] Francesco Foscarin, Jan Schlüter, and Gerhard Widmer, “Beat this! accurate beat tracking without DBN post-processing,” in *Proc. ISMIR*, 2024, pp. 962–969.
- [10] James Bolt, Johan Pauwels, and George Fazekas, “Beat and downbeat detection: A reformulated approach,” in *Proc. ICASSP*, 2026, pp. 15282–15286.
- [11] Jiajun Deng, Yaolong Ju, Jing Yang, Simon Lui, and Xunying Liu, “Efficient adapter tuning for joint singing voice beat and downbeat tracking with self-supervised learning features,” in *Proc. ISMIR*, 2024.
- [12] Ganghui Ru, Jieying Wang, Jiahao Zhao, Yulun Wu, Yi Yu, Nannan Jiang, Wei Wang, and Wei Li, “HingeNet: A harmonic-aware fine-tuning approach for beat tracking,” in *Proc. ICME*, 2025, pp. 1–6.
- [13] Ganghui Ru, Jieying Wang, Jiahao Zhao, Yulun Wu, Yi Yu, Nannan Jiang, Wei Wang, and Wei Li, “BeatFM: Improving beat tracking with pre-trained music foundation model,” in *Proc. ICME*, 2025, pp. 1–6.
- [14] Ganghui Ru, Yi Yu, and Wei Li, “BeatMamba: Bidirectional selective state-space modeling for efficient beat tracking,” in *Proc. ICASSP*, 2026.
- [15] Zhicheng Zhang, Ganghui Ru, and Wei Li, “BeatKAN: An efficient and drum-attuned beat tracking method using kolmogorov-arnold networks,” in *Proc. ICASSP*, 2025, pp. 1–5.
- [16] Daniel P. W. Ellis, “Beat tracking by dynamic programming,” *Journal of New Music Research*, vol. 36, no. 1, pp. 1–10, 2007.
- [17] Thomas Fillon, Cyril Joder, Simon Durand, and Slim Essid, “A conditional random field system for beat tracking,” in *Proc. ICASSP*, 2015, pp. 424–428.
- [18] Florian Krebs, Sebastian Böck, and Gerhard Widmer, “An efficient state-space model for joint tempo and meter tracking,” in *Proc. ISMIR*, 2015, pp. 72–78.
- [19] Jaehoon Ahn, Tae Gum Hwang, and Moon-Ryul Jung, “The SMC blind spot: A failure mode analysis of state-of-the-art beat tracking,” 2026, <https://arxiv.org/abs/2605.12287>.
- [20] Ching-Yu Chiu, Meinard Müller, Matthew E. P. Davies, Alvin Wen-Yu Su, and Yi-Hsuan Yang, “Local periodicity-based beat tracking for expressive classical piano music,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 31, pp. 2824–2835, 2023.
- [21] Zhanhong He, Hanyu Meng, Defeng Huang, and Roberto Togneri, “Joint estimation of piano dynamics and metrical structure with a multi-task multi-scale network,” in *Proc. ICASSP*, 2026, pp. 14607–14611.
- [22] Sebastian Böck and Matthew E. P. Davies, “Deconstruct, analyse, reconstruct: How to improve tempo, beat, and downbeat estimation,” in *Proc. ISMIR*, 2020, pp. 574–582.
- [23] Colin Raffel, Brian McFee, Eric J. Humphrey, Justin Salamon, Oriol Nieto, Dawen Liang, and Daniel P. W. Ellis, “mir_eval: A transparent implementation of common MIR metrics,” in *Proc. ISMIR*, 2014, pp. 367–372.

- [24] Filip Korzeniowski, Sebastian Böck, and Gerhard Widmer, “Probabilistic extraction of beat positions from a beat activation function,” in Proc. ISMIR, 2014, pp. 513–518.
- [25] Francesco Foscarin, Filip Korzeniowski, and Richard Vogl, “Masked diffusion enables coherent beat tracking,” in Proc. ISMIR, 2026.